
Dsp32

یک سیستم عامل رومیزی که روی یک تراشه‌ی چند دلاری اجرا می‌شود
و خانه را هوشمند می‌کند

این دفترچه توضیح می‌دهد Dsp32 چیست، از چه ساخته شده، چگونه کار
می‌کند، و چگونه می‌شود با آن چراغ و پرده و پمپ و سنسور یک خانه را – بدون
سرور، بدون اشتراک ماهانه و بدون اینکه چیزی از خانه بیرون برود – مدیریت
کرد.

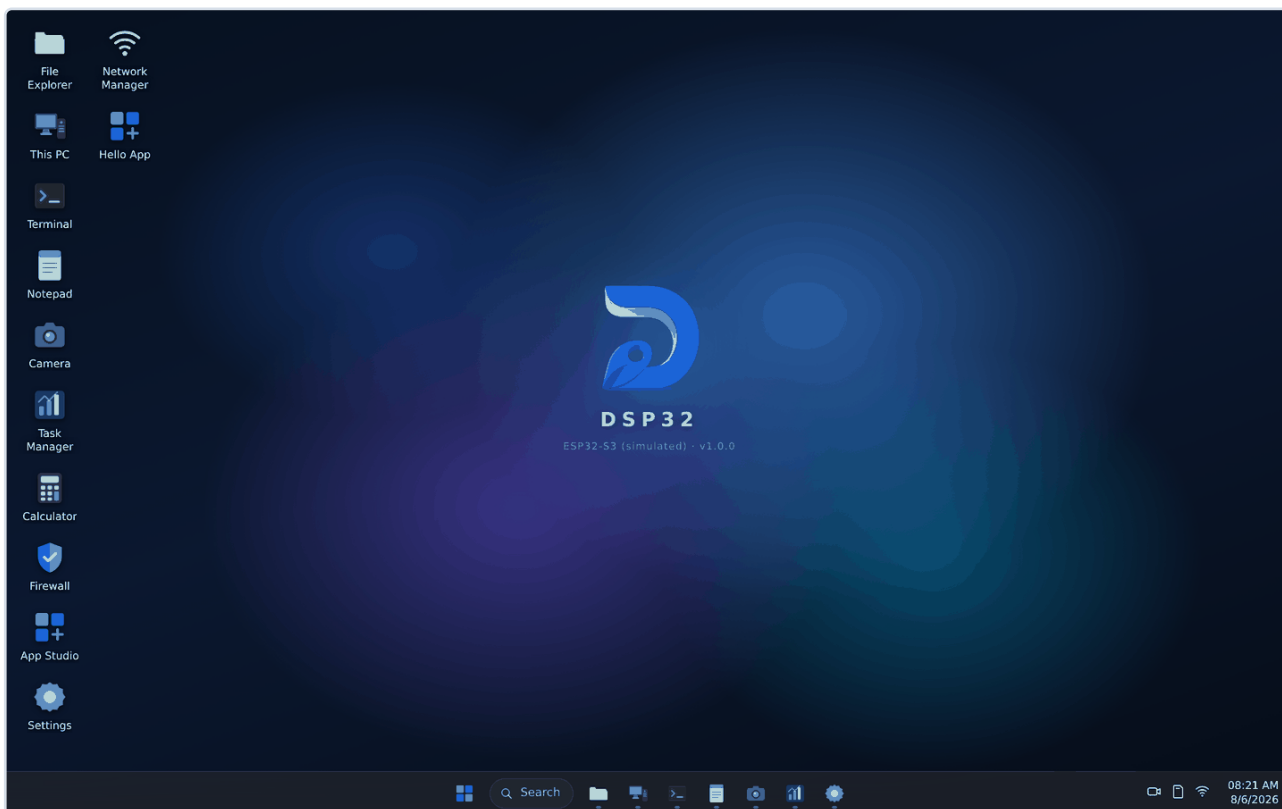
فهرست

۳	۱ – این دقیقاً چیست؟
۳	تصویر کلی، و اینکه چه چیزی نیست
۶	۲ – چرا این کار اصلاً شدنی است
۸	۳ – سخت‌افزار: چه چیزی لازم دارید
۱۰	۴ – معماری نرم‌افزار
۱۲	۵ – سامانه‌ی برنامه‌ها
۱۴	۶ – دیبا منیجر: منطق، بدون کدنویسی
۱۵	۷ – Dmesh: شبکه‌ی گره‌های ESP8266
۱۸	۸ – امنیت، بی‌تعارف
۲۰	۹ – خانه‌ی هوشمند: سناریوهای واقعی
۲۵	۱۰ – برق، باتری و مصرف
۲۷	۱۱ – راه‌اندازی گام‌به‌گام
۲۹	۱۲ – محدودیت‌ها و آنچه نباید انتظار داشت
۳۱	۱۳ – مقایسه با گزینه‌های دیگر
۳۲	پیوست – مرجع فنی

۱ این دقیقاً چیست؟

تصور کنید یک قطعه‌ی الکترونیکی به اندازه‌ی یک جعبه‌ی کبریت و به قیمت چند دلار را به برق می‌زنید. چند ثانیه بعد، یک شبکه‌ی وای‌فای به نام Dibandain ظاهر می‌شود. با گوشی به آن وصل می‌شوید و مرورگر را باز می‌کنید – و یک میز کار کامل می‌بینید: نوار وظیفه، منوی شروع، پنجره‌هایی که جابه‌جا و بزرگ و کوچک می‌شوند، مدیر فایل، پایانه، فروشگاه برنامه.

هیچ چیزی نصب نکرده‌اید. هیچ سروری در کار نیست. هیچ اشتراکی نخریده‌اید. آنچه می‌بینید از خود همان قطعه می‌آید – و همان قطعه، در همان لحظه، می‌تواند چراغ پذیرایی را روشن کند، دمای اتاق خواب را بخواند و اگر رطوبت گلدان کم شد پمپ را برای ده ثانیه به کار بیندازد.



میز کار، همان‌طور که در مرورگر گوشی یا لپ‌تاپ دیده می‌شود. تمام این تصویر از یک تراشه‌ی ESP32 سرو می‌شود.

در یک جمله

Dsp32 یک میان‌افزار اختصاصی برای خانواده‌ی تراشه‌های ESP32 است که یک سیستم‌عامل رومیزی کامل را روی وای‌فای خودش سرو می‌کند، و از همان‌جا سخت‌افزار خانه را مدیریت می‌کند.

سه بخش در این جمله هست که هرکدام را در فصل‌های بعد باز می‌کنیم:

- **میان‌افزار اختصاصی** – کد به زبان C و روی ESP-IDF نوشته شده، نه آردوینو. این تفاوت، تفاوت دسترسی به سخت‌افزار در پایین‌ترین سطح ممکن است: مدیریت حافظه، تردها، خواب سبک، فرکانس پردازنده.
- **سیستم‌عامل رومیزی** – نه یک صفحه‌ی تنظیمات، بلکه یک محیط با پنجره، چندوظیفگی، سامانه‌ی فایل، و برنامه‌هایی که کاربر خودش نصب و حذف می‌کند.
- **مدیریت سخت‌افزار خانه** – پین‌های خود برد، و از طریق پروتکل Dmesh، هر تعداد گرهی ESP8266 که در گوشه‌وکنار خانه گذاشته‌اید.

چه چیزی نیست

صادق بودن درباره‌ی مرزها، به اندازه‌ی توضیح توانایی‌ها مهم است. اگر انتظار اشتباه داشته باشید، تجربه‌ی بدی خواهید داشت – و تقصیر شما نیست.

این یک کامپیوتر نیست

مرورگر وب ندارد که با آن اینترنت بگردید. ویدیوی ۴K پخش نمی‌کند. نمی‌شود روی آن بازی سنگین اجرا کرد. کل حافظه‌ی در دسترسش حدود ۳۲۰ کیلوبایت است – یعنی تقریباً یک‌هزارم حافظه‌ی یک گوشی معمولی. آنچه دیده می‌شود، یک رابط کاربری است که در مرورگر شما اجرا می‌شود و برد فقط آن را می‌فرستد و به درخواست‌هایش جواب می‌دهد.

این تقسیم کار، هوشمندانه‌ترین بخش طراحی است و در فصل ۲ کامل توضیح داده می‌شود. اما نتیجه‌ی عملی‌اش را همین‌جا بگویم: **سنگینی کار روی گوشی یا لپ‌تاپ شماست، نه روی برد.** برد فقط کارهایی را می‌کند که واقعاً فقط از او برمی‌آید – خواندن یک ولتاژ، روشن کردن یک رله، حرف زدن با یک گرهی دیگر.

برای چه کسی ساخته شده

اگر شما...	Dsp32 برای شما...
می‌خواهید خانه‌تان هوشمند شود ولی نمی‌خواهید داده‌هایتان روی سرور شرکتی برود	مناسب است. هیچ چیزی از خانه بیرون نمی‌رود مگر خودتان بخواهید. اینترنت هم لازم نیست.
الکترونیک‌کار یا برنامه‌نویس هستید و می‌خواهید چیزی بسازید که واقعاً مال خودتان باشد	مناسب است. کل سامانه‌ی برنامه‌ها باز است؛ می‌توانید برنامه‌ی خودتان را بنویسید و نصب کنید.
دنبال محصولی هستید که از جعبه دربیايد و بدون هیچ دانشی کار کند	هنوز نه. راه‌اندازی اولیه سیم‌کشی و کمی حوصله می‌خواهد.
می‌خواهید صدها دستگاه در یک ساختمان اداری مدیریت کنید	احتمالاً نه. این برای یک خانه ساخته شده، نه برای یک ساختمان.

چرا اسمش این است

Dsp32 از دو تکه ساخته شده: Dsp اشاره‌ی کوتاهی به دیبا (سازنده) دارد و 32 همان ESP32 است. گره‌های کوچک‌تر هم Di8266 نام گرفته‌اند – همان الگو، روی تراشه‌ی ESP8266.

پیش از ادامه، یک بار امتحانش کنید

لازم نیست چیزی بخرید تا ببینید از چه حرف می‌زنیم. نسخه‌ی شبیه‌سازی‌شده‌ی کامل سیستم در مرورگر اجرا می‌شود: میز کار همان میز کار واقعی است و دستگاه پشت آن داخل خود صفحه شبیه‌سازی می‌شود.

<https://aliakrami1375.github.io/Dsp32-firmware/demo/>

۲ چرا این کار اصلاً شدنی است

یک ESP32 حدود ۳۲۰ کیلوبایت حافظه‌ی قابل استفاده دارد. یک صفحه‌ی وب معمولی امروز چند مگابایت است. پس چطور یک میز کار کامل روی چنین قطعه‌ای اجرا می‌شود؟ پاسخ، در یک جمله: **اجرا نمی‌شود**.

تقسیم کاری که همه چیز را ممکن می‌کند

میز کار – پنجره‌ها، انیمیشن‌ها، منوها، همه‌ی زیبایی‌اش – کدی است که در مرورگر شما اجرا می‌شود. برد فقط دو کار می‌کند: آن کد را یک بار می‌فرستد، و بعد به درخواست‌های کوچکی جواب می‌دهد که آن کد برایش می‌فرستد.



این درخواست‌ها کوچک‌اند. «دمای پین شماره ۴ چقدر است؟» یک بسته‌ی چندصد بایتی است و پاسخش کوچک‌تر. «فهرست فایل‌های این پوشه» شاید یک کیلوبایت. برد هیچ‌وقت لازم نیست یک صفحه‌ی کامل بسازد، چون صفحه از قبل در مرورگر است و فقط بخش کوچکی از آن به‌روز می‌شود.

حساب حافظه

کل رابط کاربری – همه‌ی کدهای جاوااسکریپت، شیوه‌نامه‌ها، آیکون‌ها – فشرده حدود ۱۱۴ کیلوبایت است و به‌صورت فشرده داخل خود میان‌افزار جا می‌گیرد. برد آن را یک بار می‌فرستد و مرورگر در حافظه‌ی خودش نگهش می‌دارد.

بخش	اندازه	کجا
میان‌افزار کامل (کد C + رابط کاربری فشرده)	~۱/۳۶ مگابایت	فلش برد
پارتیشن برنامه	۲/۵ مگابایت	فلش برد – حدود ۴۷٪ خالی
حافظه‌ی آزاد هنگام کار	~۱۸۰ کیلوبایت	RAM برد
برنامه‌های نصب‌شدنی	۳ تا ۲۲ کیلوبایت هرکدام	دانلود می‌شوند، همراه میان‌افزار نیستند

هیچ برنامه‌ای داخل میان‌افزار نیست

این یک تصمیم طراحی است، نه یک محدودیت. اگر هشت برنامه داخل میان‌افزار بودند، هر کاربر مجبور بود هزینه‌ی فضای هر هشت‌تا را بدهد حتی اگر یکی را هم نخواهد. به‌جایش، برد فقط آنچه را می‌خواهید دانلود می‌کند – و همین است که یک تصویر ۱/۳ مگابایتی می‌تواند اصلاً یک میز کار داشته باشد.

چرا آردوینو نه

آردوینو برای شروع سریع عالی است، اما لایه‌ای بین شما و سخت‌افزار می‌گذارد که برای این کار گران تمام می‌شود. این پروژه روی ESP-IDF نوشته شده – همان چارچوبی که خود سازنده‌ی تراشه استفاده می‌کند. تفاوت‌ها در عمل:

- **تردهای واقعی.** مدیر وظایف، فهرست واقعی تردهای FreeRTOS را نشان می‌دهد: نام، وضعیت، فضای پشته‌ی باقی‌مانده، و سهم هر ترد از پردازنده.
- **مدیریت توان.** فرکانس پردازنده از تنظیمات قابل تغییر است و خواب سبک را می‌شود روشن کرد – چیزی که در فصل ۱۰ برای عمر باتری حیاتی می‌شود.
- **کنترل حافظه.** وقتی ۳۲۰ کیلوبایت دارید، باید بدانید هر بایت کجا می‌رود. جریان‌سازی فایل‌ها در قطعات ۴ کیلوبایتی یعنی یک فیلم دو گیگابایتی همان‌قدر حافظه می‌خواهد که یک فایل متنی.

۳ سخت‌افزار: چه چیزی لازم دارید

کمترین چیزی که برای شروع لازم است، یک برد ESP32 و یک کابل USB است. همین. بقیه – کارت حافظه، دوربین، رله، سنسور – هرکدام قابلیت اضافه می‌کنند و هیچ‌کدام اجباری نیستند.

بردهای پشتیبانی‌شده

برد	هسته	دوربین	توضیح
ESP32	۲	با ماژول	نسخه‌ی اصلی. ارزان و همه‌جا پیدا می‌شود.
ESP32-CAM	۲	دارد	دوربین و کارت‌خوان روی خود برد. برای دوربین در خانه.
ESP32-S2	۱	با ماژول	USB بومی، بدون بلوتوث.
ESP32-S3	۲	با ماژول	بهترین انتخاب. دو هسته و PSRAM.
XIAO ESP32S3 Sense	۲	دارد	خیلی کوچک، با دوربین و کارت‌خوان.
ESP32-C3	۱	ندارد	معماری RISC-V، ارزان‌ترین گزینه.
ESP32-C6	۱	ندارد	با Wi-Fi 6.

اگر یکی می‌خرید

ESP32-S3 با ۸ مگابایت PSRAM بگیرید. دو هسته دارد، حافظه‌ی اضافه‌اش کار با دوربین و فایل‌های بزرگ را راحت می‌کند، و همه‌ی قابلیت‌های این دفترچه رویش کار می‌کند. تفاوت قیمتش با ارزان‌ترین گزینه معمولاً کمتر از دو دلار است.

افزودنی‌ها

کارت حافظه – مهم‌ترین افزودنی

فلش داخلی برد چند مگابایت است و میان‌افزار هم داخلش هست. برای نگه داشتن عکس‌های دوربین، فایل‌های صوتی، یا فقط برای اینکه نصب برنامه‌ها فلش داخلی را پر نکند، یک کارت حافظه تفاوت بزرگی می‌سازد.

سیم‌کشی کارت در Dsp32 **تنظیم زمان اجراست، نه زمان ساخت** – یعنی یک میان‌افزار روی بردهایی با کارت‌خوان‌های متفاوت کار می‌کند. اگر برد شما از سیم‌کشی‌های شناخته‌شده باشد، خودش پیدایش می‌کند؛ وگرنه پین‌ها را در **تنظیمات** ← حافظه وارد می‌کنید.

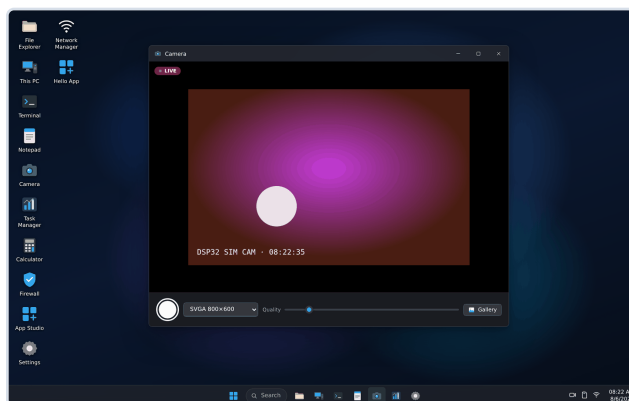
دوربین

میان‌افزار سیم‌کشی دوربین را **خودش تشخیص می‌دهد**. هفت سیم‌کشی شناخته‌شده را یکی‌یکی امتحان می‌کند و اولی را که یک فریم کامل بدهد نگه می‌دارد و به یاد می‌سپارد. اگر بردتان در فهرست نبود، شانزده پینش را در تنظیمات وارد می‌کنید و بلافاصله امتحان می‌شود.

روی تراشه‌هایی که اصلاً رابط دوربین ندارند – مثل C3 و C6 – برنامه‌ی دوربین صریح می‌گوید «پشتیبانی نمی‌شود»، نه اینکه یک صفحه‌ی خالی نشان بدهد.



مدیر وظایف با نمودار زنده‌ی حافظه



نمای زنده و ثبت عکس

معماری نرم افزار ۴

سه لایه روی هم: میان افزار که سخت افزار را می راند، یک واسط REST که آن را در دسترس می گذارد، و میز کاری که در مرورگر اجرا می شود و فقط با همان واسط حرف می زند.

لایه اول – میان افزار

حدود ده هزار خط C که هرکدام یک مسئولیت مشخص دارند:

پیمانه	مسئولیت
dsp32_wifi	اکسس پوینت، اتصال به شبکه ی بالادست، فایروال مبتنی بر MAC
dsp32_fs	فلش داخلی و کارت SD، با تشخیص خودکار سیم کشی کارت
dsp32_http	سرور وب و ۶۶ مسیر API
dsp32_gpio	نقشه ی توانایی پین ها، ورودی/خروجی، آنالوگ، PWM
dsp32_camera	تشخیص خودکار سیم کشی، جریان MJPEG روی پورت جدا
dsp32_mesh	ثبت گره ها، کلیدها، وضعیت مطلوب، هماهنگ سازی
dsp32_battery	پایش باتری در ترد جداگانه با اولویت پایین
dsp32_media	سرور رسانه روی پورت مستقل
dsp32_flasher	فلش کردن ESP8266 خام از طریق پروتکل بوت لودر

ترتیب بالا آمدن، و چرا مهم است

ترتیب راه اندازی این پیمانه ها تصادفی نیست. اکسس پوینت **تنها راه برگشت** به بردی است که خراب شده – اگر چیزی قبل از آن گیر کند، نه میز کاری هست، نه لاگی که از راه دور خوانده شود، و تنها راه، فلش دوباره است.

```
// حافظه و پین ها – بقیه ممکن است لازم شان داشته باشند
dsp32_fs_init(); dsp32_sys_init(); dsp32_gpio_init();

// راه ورود، قبل از هر چیزی که ممکن است بد رفتار کند
dsp32_wifi_init(); dsp32_dns_start(); dsp32_http_start();

// بقیه – حالا هیچ کدام نمی توانند میز کار را زمین بزنند
dsp32_camera_init(); dsp32_battery_init(); dsp32_mesh_init();
dsp32_flash8266_init(); dsp32_media_init();
```

این درس گران تمام شد

در نسخه‌ی ۱/۱/۰ همین ترتیب برعکس بود و ترد Dmesh قبل از بالا آمدن پشت‌پشتی شبکه یک سوکت باز می‌کرد. برد قبل از رسیدن به رادیو ری‌استارت می‌شد – یعنی هیچ اکسس‌پوینتی روشن نمی‌شد و هیچ راهی جز فلش دوباره نبود. حالا هر مرحله پیش از اجرا نامش را روی خروجی سریال چاپ می‌کند تا اگر جایی گیر کرد، معلوم باشد کجا.

لایه‌ی دوم – واسط REST

هر کاری که می‌ز کار می‌کند، از طریق یک درخواست HTTP ساده انجام می‌شود. این یعنی هر چیزی که می‌ز کار می‌تواند بکند، شما هم می‌توانید – از پایانه، از یک اسکریپت، از یک برنامه‌ی دیگر.

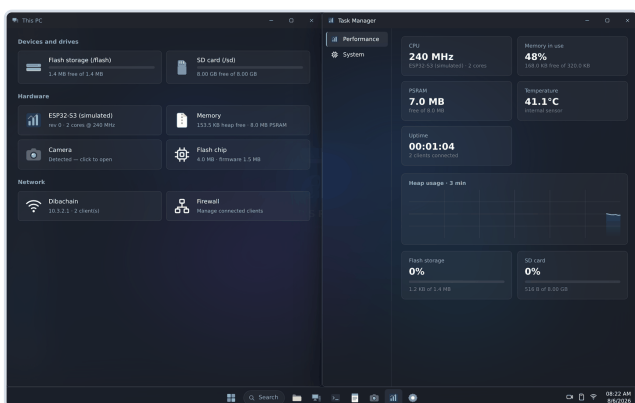
```
# خواندن دمای پردازنده و حافظه‌ی آزاد
curl http://۱۰.۳.۲.۱/api/system

# روشن کردن یک خروجی
curl -X POST "http://۱۰.۳.۲.۱/api/gpio/write?pin=۴&value=۱"

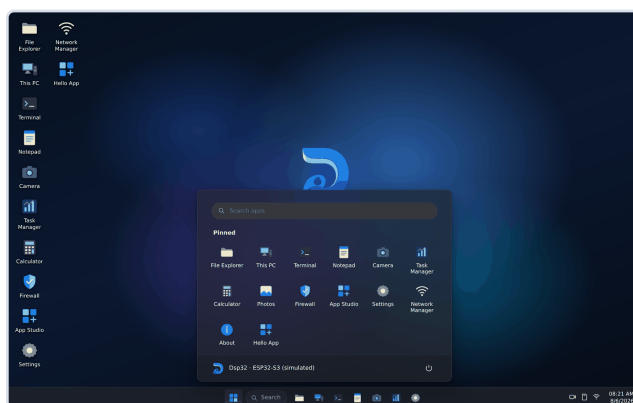
# خواندن یک ورودی آنالوگ
curl "http://۱۰.۳.۲.۱/api/gpio/read?pin=۱"
```

لایه‌ی سوم – میز کار

جاوااسکریپت خالص، بدون هیچ کتابخانه‌ی بیرونی. این تصمیم عمدی است: هر کتابخانه‌ای که اضافه می‌شد، حجمش را باید داخل فلش برد جا می‌دادیم.



چسباندن پنجره‌ها به لبه‌ها



منوی شروع، قابل جست‌وجو

۵ سامانه‌ی برنامه‌ها

یک برنامه در Dsp32 یک بسته‌ی **dib** است – یک فایل که همه‌ی چیزهای لازم را دارد و کاربر خودش نصب و حذفش می‌کند. هیچ برنامه‌ای همراه میان‌افزار نمی‌آید.

ساختار یک بسته

محتوا | JSON | طول مانیفست (۴ بایت) | مانیفست | "DIB1"

مانیفست می‌گوید برنامه چه نام دارد، چه نسخه‌ای است، کدام فایل نقطه‌ی شروع است، و – مهم‌تر از همه – چه مجوزهایی می‌خواهد. کاربر پیش از نصب همه‌ی آن‌ها را می‌بیند.

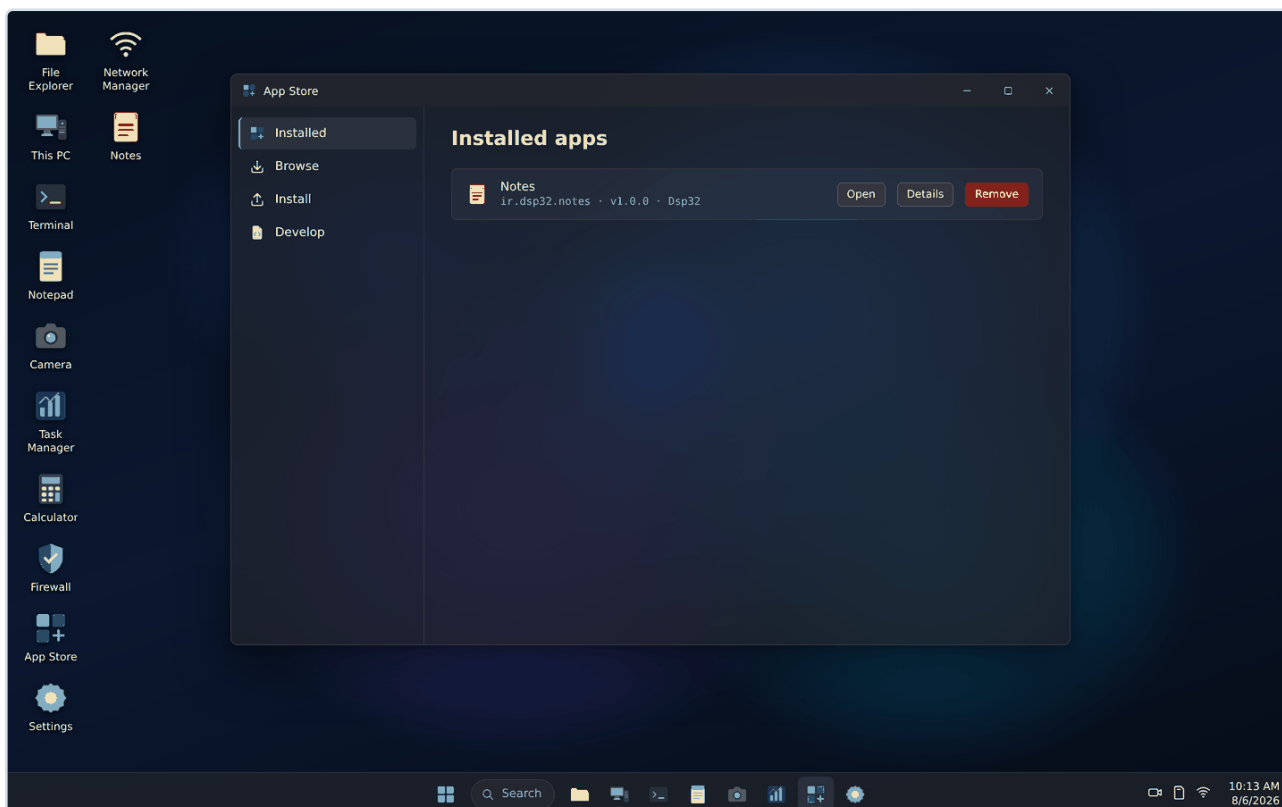
مجوز	چه چیزی می‌دهد
storage	یک انبار کلید-مقدار خصوصی روی دستگاه
fs	خواندن و نوشتن در فلش و کارت حافظه
net	درخواست اینترنتی، با واسطه‌ی خود دستگاه
notify	اعلان روی میز کار
system	اطلاعات تراشه، حافظه و سنسورها
camera	عکس گرفتن

این مجوزها یک قرنطینه‌ی امنیتی نیستند

یک برنامه‌ی نصب‌شده، جاوااسکریپت است که در همان صفحه‌ی میز کار اجرا می‌شود؛ پس یک برنامه‌ی بدخواه به هر چیزی که میز کار دسترسی دارد می‌رسد، با مجوز یا بدون آن. جداسازی واقعی نیاز به اجرای هر برنامه در یک قاب مجزا دارد که این نسخه ندارد. فهرست مجوزها می‌گوید یک برنامه‌ی صادق قصد دارد از چه استفاده کند – نه بیشتر. بسته‌هایی را نصب کنید که به منبعشان اعتماد دارید.

برنامه‌های موجود

برنامه	کار
مدیر فایل	فلش و کارت، آپلود با کشیدن و رها کردن، تغییر نام، حذف
مدیر وظایف	تردهای واقعی FreeRTOS، نمودار زنده‌ی حافظه، دما
پایانه	دستورهای متنی روی همان واسط REST
دیبا منیجر	ساخت منطق با کشیدن نود – فصل ۶
Dmesh	مدیریت گره‌های ESP8266 – فصل ۷
سرور رسانه	سرو کردن یک پوشه‌ی کارت به صورت کتابخانه‌ی وب
فایروال	کنترل اینکه چه دستگاهی به شبکه‌ی برد وصل شود
ویرایشگر کد	نوشتن برنامه روی خود دستگاه



فروشگاه برنامه – نصب از مخزن، از کارت حافظه، یا از یک فایل

۶ دیبا منیجر: منطق، بدون کدنویسی

اگر خواستید «وقتی رطوبت خاک کمتر از فلان شد، پمپ را ده ثانیه روشن کن»، لازم نیست کد بنویسید. در دیبا منیجر نودها را کنار هم می‌گذارید و به هم وصل می‌کنید.

نودهای موجود

گروه	نودها
ورودی	پین ورودی، ورودی آنالوگ، تایمر دوره‌ای، مقدار ثابت، ساعت
شبکه‌ی گره	خواندن پین یک گره، نوشتن روی پین یک گره
اینترنت	درخواست HTTP، انتخاب فیلد از پاسخ
منطق	اگر/وگرنه با چند شرط، مقایسه، نقیض، و، یا، ریاضی، نگاهداشتن، حلقه، میانگین
خروجی	پین خروجی، PWM، اعلان، گیج، ویجت میز کار، ثبت در فایل

نمونه: آبیاری خودکار



Deploy – اجرای دائمی

یک جریان دو حالت دارد. اجرا برای وقتی است که دارید می‌سازید و می‌خواهید ببینید مقادیر چطور روی سیم‌ها حرکت می‌کنند. **Deploy** آن را به پس‌زمینه می‌سپارد: پنجره را ببندید، صفحه را رفرش کنید، از خانه بروید بیرون – جریان همچنان اجرا می‌شود و در مدیر وظایف و گوشه‌ی میز کار دیده می‌شود.

ESP8266: شبکه‌ی گره‌های Dmesh v

Dsp32 روی یک برد اجرا می‌شود. Dmesh راهی است که با آن بردهای دیگر را می‌رانند. یک گره‌ی Di8266 یک ESP8266 است که پین‌هایش را در اختیار برد اصلی می‌گذارد – و همین است که یک برد را به کنترل‌کننده‌ی کل خانه تبدیل می‌کند.

چرا یک پروتکل اختصاصی

یک گره قطعه‌ای ۸۰ مگاهرتزی با حدود ۴۰ کیلوبایت حافظه‌ی قابل استفاده است. HTTP برایش یعنی یک اتصال TCP، پارس هدر و یک سوکت به ازای هر تبادل؛ کشف با mDNS یعنی یک شنونده‌ی multicast که باید همیشه بالا باشد. هیچ‌کدام روی قطعه‌ای که باید تغییر یک پین را زیر یک میلی‌ثانیه جواب بدهد به‌صرفه نیست.

Dmesh هر پیام را در یک بسته‌ی UDP می‌فرستد. گره اصلاً حالت اتصال نگه نمی‌دارد و کل پیاده‌سازی زیر ۳۰۰ خط است.

دو ایده‌ای که کل طراحی از آن‌ها می‌آید

یک – شما یک گره را پیکربندی می‌کنید، نه یک اتصال

آنچه گره باید باشد، همان لحظه‌ای که می‌خواهید نوشته می‌شود – چه وصل باشد چه نه. دفعه‌ی بعد که گره خبر داد، برد می‌بیند وضعیتش آن چیزی نیست که خواسته شده و درستش می‌کند.

نتیجه‌ی عملی: **گره‌ی خاموش یک خطا نیست**. اگر گره‌ای در انباری برق نداشت و شما تنظیماتش را عوض کردید، هیچ پیام خطایی نمی‌بینید و هیچ صفی از دستوره‌های معلق ساخته نمی‌شود. گره فقط «هماهنگ نیست» و برنامه دقیقاً همین را می‌گوید. برق که وصل شد، ظرف یک ضربان اصلاح می‌شود.

دو – گره‌ها خبر می‌دهند، کنترل‌کننده نمی‌پرسد

هر گره هر هشت ثانیه یک ضربان می‌فرستد که در آن یک عدد ۳۲ بیتی از وضعیت خودش هست. تصمیم اینکه «این گره چیزی لازم دارد؟» یک مقایسه‌ی عدد صحیح است.

برای همین تعداد گره مهم نیست

نه تایم‌ری به ازای گره هست، نه سوکتی، نه اسکنی که فهرست را بپیماید. گره صدم فقط یک ضربان بیشتر در هر هشت ثانیه هزینه دارد. اگر کنترل‌کننده از هر گره می‌پرسید، پنجاه گره یعنی پنجاه رفت‌وبرگشت با پنجاه انتظار – دقیقاً برعکس جهتی که باید مقیاس بگیرد.

قوانین: کاری که خود گره انجام می‌دهد

یک قانون، وظیفه‌ای است که به گره سپرده می‌شود و روی خود گره اجرا می‌شود. این مهم است: آستانه‌ای که باید قبل از عمل از برد اجازه بگیرد، زمان واکنشش زمان شبکه است. قوانین با خاموش بودن برد اصلی هم کار می‌کنند و از ری‌استارت گره جان سالم به در می‌برند.

نوع	کار
آستانه	تا وقتی پین دیده‌شده بالاتر/پایین‌تر از مقداری است، پین دیگری را بران. با زمان نگه‌داشتن، تبدیل به یک پالس می‌شود.
لبه	فقط یک بار، در لحظه‌ی گذار به شرط – نه تا وقتی شرط برقرار است.
گزارش	وقتی مقدار بیش از حد مشخصی جابه‌جا شد، به برد خبر بده.
چشمک	پین را روی یک تایمر خاموش‌روشن کن – مثلاً یک LED که ثابت می‌کند گره زنده است.

یک قانون تا وقتی غیرفعال یا حذفش نکنید هست. این یک وظیفه است، نه یک دستور.

راه‌اندازی یک گره

یک ESP8266 خام نه کشف‌شدنی است و نه از راه دور به‌روز می‌شود، چون فرمویری ندارد که این کارها را بکند. اما همه‌ی آن‌ها بوت‌لودر را در حافظه‌ی ثابت دارند – پاک‌شدنی نیست و روی تراشه‌ای که هرگز برنامه‌ریزی نشده هم هست.

از داخل برنامه‌ی Dmesh، با **چهار سیم** و بدون هیچ کامپیوتری:

```
Dsp32 TX → ESP8266 RX
Dsp32 RX ← ESP8266 TX
Dsp32 pin → ESP8266 GPIO0
Dsp32 pin → ESP8266 RST
GND      – GND
```

دو پین GPIO0 و RST هستند که این کار را ممکن می‌کنند: پایین نگه داشتن GPIO0 در طول یک ری‌استارت، تراشه را به بوت‌لودر می‌برد – کاری که بدون آن دو سیم فقط با دست شدنی است.

بعد از آن، همه چیز از راه دور است. گره تازه، شبکه‌ای به نام Di8266 با رمز 123@Test بالا می‌آورد و خودش را اعلام می‌کند.

پین‌های یک گره

پین	دیجیتال	PWM	آنالوگ	کاربرد معمول
D1–D8	✓	✓	–	رله، LED، کلید، سنسور حرکت
A0	–	–	✓	رطوبت خاک، نور، ولتاژ

پین‌ها با همان نامی که روی برد چاپ شده نشان داده می‌شوند، نه با شماره‌ی خام GPIO – چون هیچ‌کس با نگاه به یک NodeMCU «GPIO ۴» نمی‌بیند.

Dmesh و دیبا منیجر

گره‌های تصاحب‌شده در دیبا منیجر به‌عنوان نود ظاهر می‌شوند. یعنی یک جریان می‌تواند سنسوری را روی گره‌ی گلخانه بخواند و پمپی را روی گره‌ی کنار مخزن بران – بدون یک خط کد.

۸ امنیت، بی‌تعارف

این فصل هم می‌گوید چه چیزی محافظت می‌شود و هم صریح می‌گوید چه چیزی نمی‌شود. دومی مهم‌تر است: اگر انتظار اشتباه داشته باشید، تصمیم اشتباه می‌گیرید.

آنچه واقعاً محافظت می‌شود

ارتباط با گره‌ها

هر گره کلید ۱۶ بیتی مخصوص خودش را دارد که هنگام تصاحب از مولد تصادفی سخت‌افزاری برد ساخته می‌شود. هیچ دو گره‌ای کلید یکسان ندارند و کلیدها هرگز از برد بیرون نمی‌روند.

```
enc = SHA256(master || "dmesh-enc-v2")[0..15]
mac = SHA256(master || "dmesh-mac-v2")[0..15]
ct = AES-128-CTR(enc, nonce = bootId || counter, plaintext)
tag = HMAC-SHA256(mac, header || ct)[0..7]
```

اول رمزنگاری، بعد امضا – تا بسته‌ی جعلی پیش از آنکه چیزی رمزگشایی شود دور انداخته شود. شمارنده هم nonce است و هم نگهبان تکرار: گیرنده بالاترین مقداری را که دیده به یاد دارد و هر چیز برابر یا کمتر را می‌اندازد.

فایروال شبکه

می‌توانید مشخص کنید چه دستگاه‌هایی اجازه‌ی اتصال به شبکه‌ی برد را دارند، بر اساس نشانی MAC. حالت «فهرست مجاز» یعنی هر چیزی که در فهرست نیست، وصل نمی‌شود.

آنچه محافظت نمی‌شود

سه چیز را باید بدانید

۱ – ارتباط با خود برد رمزنگاری نشده است. میز کار روی HTTP ساده سرو می‌شود، نه HTTPS. کسی که روی همان شبکه‌ی وای‌فای باشد و ترافیک را گوش بدهد، می‌تواند بخواند. محافظت واقعی، همان رمز WPA2 شبکه است.

۲ – رمز ورود سیستم، قفل کشو است نه گاوصندوق. رمز با salt و ۶۰ هزار دور SHA-256 ذخیره می‌شود تا از روی فلش لو نرود، اما چون HTTPS نیست، هنگام ورود در هوا واضح می‌رود. و هر کسی که برد را در دست داشته باشد می‌تواند فلش را بخواند.

۳ – لحظه‌ی تصاحب یک گره، نقطه‌ی ضعف است. کلید یک بار به صورت واضح تحویل می‌شود. دو چیز محدودش می‌کند: گره فقط در دو دقیقه‌ی اول بعد از روشن شدن تصاحب را می‌پذیرد، و گره‌ای که کلید دارد کلید دیگری نمی‌گیرد.

توصیه‌ی عملی

- رمز اکسس‌پوینت برد را از پیش‌فرض عوض کنید. این مهم‌ترین کاری است که می‌توانید بکنید.
- گره‌ها را روی شبکه‌ای تصاحب کنید که در اختیار خودتان است، نه روی وای‌فای عمومی.
- اگر برد را به اینترنت وصل می‌کنید، پورتش را از بیرون باز نگذارید.
- برای سرور رسانه، اگر لینک قرار است از خانه بیرون برود، «اجازه‌ی تغییر» را خاموش کنید تا فقط خواندنی باشد.

۹ خانه‌ی هوشمند: سناریوهای واقعی

تا اینجا گفتیم سیستم چیست و چطور کار می‌کند. این فصل نشان می‌دهد با آن چه می‌شود کرد – با سیم‌کشی مشخص، تنظیمات مشخص، و توضیح اینکه هر تصمیم چرا گرفته شده.

معماری یک خانه

الگوی درست این است: یک برد ESP32 به‌عنوان مغز، و هر تعداد گره‌ی ESP8266 در نقاطی که کاری باید انجام شود. برد اصلی جایی می‌ماند که پوشش وای‌فای خوبی به کل خانه بدهد – معمولاً وسط خانه یا کنار روتر.

برد اصلی Dsp32

مغز خانه – میز کار و قوانین
پذیرایی، کنار روتر

Dmesh روی UDP – رمزنگاری‌شده، هر گره با کلید خودش

گره – حیاط

رطوبت خاک، پمپ

گره – آشپزخانه

نشت آب، شیر برقی

گره – انباری

دما، رطوبت

گره – پذیرایی

چراغ، پرده

و جدا از این زنجیره:

گوشی شما

مرورگر – بدون نصب هیچ اپی

یک مغز، چند گره. گوشی فقط برای دیدن و تنظیم کردن است – قوانین روی خود سخت‌افزار اجرا می‌شوند.

نکته‌ای که این معماری را از بقیه جدا می‌کند

گوشی شما بخشی از زنجیره‌ی کنترل نیست. اگر گوشی خاموش باشد، اگر اینترنت قطع باشد، اگر شما در سفر باشید – چراغ‌ها همچنان با کلید دیوار کار می‌کنند، پمپ همچنان با رطوبت خاک روشن می‌شود، و آژیر نشت آب همچنان شیر را می‌بندد. گوشی فقط پنجره است.

سناریو ۱ – آبیاری خودکار گل‌دان‌ها

مسئله: چند گل‌دان که در سفر خشک می‌شوند.

قطعات

قطعه	توضیح
گره‌ی ESP8266	یک NodeMCU معمولی
سنسور رطوبت خاک	نوع خازنی بخیرید، نه مقاومتی – مقاومتی در چند هفته می‌پوسد
پمپ ۵ ولت + ماژول رله	پمپ آکواریومی کوچک کافی است
آداپتور ۵ ولت	حداقل ۱ آمپر

سیم‌کشی

(خروجی آنالوگ سنسور) A0 --> سنسور رطوبت
(ورودی فرمان رله) D1 --> رله‌ی پمپ
مشترک GND و تغذیه ۵V <-- 5

تنظیم در Dmesh

1. گره را تصاحب کنید (دکمه‌ی Claim).
2. پین A0 را روی آنالوگ بگذارید.
3. پین D1 را روی خروجی بگذارید.
4. یک قانون از نوع آستانه بسازید:

فیلد	مقدار
پین دیده‌شده	A0
شرط	کمتر از ۱۲۰۰
پین هدف	D1
مقدار	۱ (روشن)
نگه‌داشتن	۱۰۰۰۰ میلی‌ثانیه (۱۰ ثانیه)

چرا «نگه‌داشتن» مهم است

اگر زمان نگه‌داشتن را صفر بگذارید، پمپ تا وقتی خاک خشک است روشن می‌ماند – و چون آب چند دقیقه طول می‌کشد تا به سنسور برسد، گلدان را غرق می‌کند. با ده ثانیه، پمپ یک پالس کوتاه می‌زند، بعد صبر می‌کند تا خاک واقعاً خیس شود و دوباره ارزیابی کند.

عدد ۱۲۰۰ را از کجا بیاورم؟ سنسور را در خاک خشک بگذارید و مقدار A0 را در Dmesh ببینید؛ بعد خاک را آب بدهید و دوباره ببینید. آستانه را جایی وسط این دو بگذارید. هر سنسور و هر خاکی متفاوت است.

سناریو ۲ – تشخیص نشت آب و بستن شیر

مسئله: نشت زیر سینک یا کنار ماشین لباسشویی که تا دیر نشود دیده نمی‌شود.

سیم‌کشی

(خروجی دیجیتال، فعال با آب) D2 --> سنسور نشت آب
(از طریق رله) D3 --> شیر برقی
D4 --> بوق هشدار

سه قانون روی گره

نوع	شرط	عمل
لبه	D2 برابر ۱ شد	D3 = ۱ – شیر را ببند (بدون نگه‌داشتن، یعنی قفل بماند)
لبه	D2 برابر ۱ شد	D4 = ۱ به مدت ۳۰ ثانیه – بوق
گزارش	D2 تغییر کرد	به برد اصلی خبر بده

چرا «لبه» و نه «آستانه»

لبه فقط در لحظه‌ی گذار یک بار عمل می‌کند. یعنی شیر بسته می‌شود و بسته می‌ماند حتی اگر سنسور بعداً خشک شود – که دقیقاً همان چیزی است که می‌خواهید. با آستانه، به محض خشک شدن سنسور شیر دوباره باز می‌شد و نشت از سر گرفته می‌شد.

قانون سوم باعث می‌شود برد اصلی خبردار شود و بتواند اعلان بدهد یا در فایل ثبت کند. اما توجه کنید: **دو قانون اول به برد اصلی نیازی ندارند.** اگر برق برد اصلی رفته باشد یا خراب شده باشد، شیر باز هم بسته می‌شود – چون قانون روی خود گره اجرا می‌شود.

سناریو ۳ – روشنایی پله با تشخیص حرکت

مسئله: پله‌ی تاریک شب، و چراغی که همیشه یادتان می‌رود خاموش کنید.

D5 --> PIR سنسور
D6 --> چراغ (رله)

نوع	شرط	عمل
آستانه	D5 برابر ۱	D6 = ۱ با نگره داشتن ۱۲۰۰۰۰ (دو دقیقه)

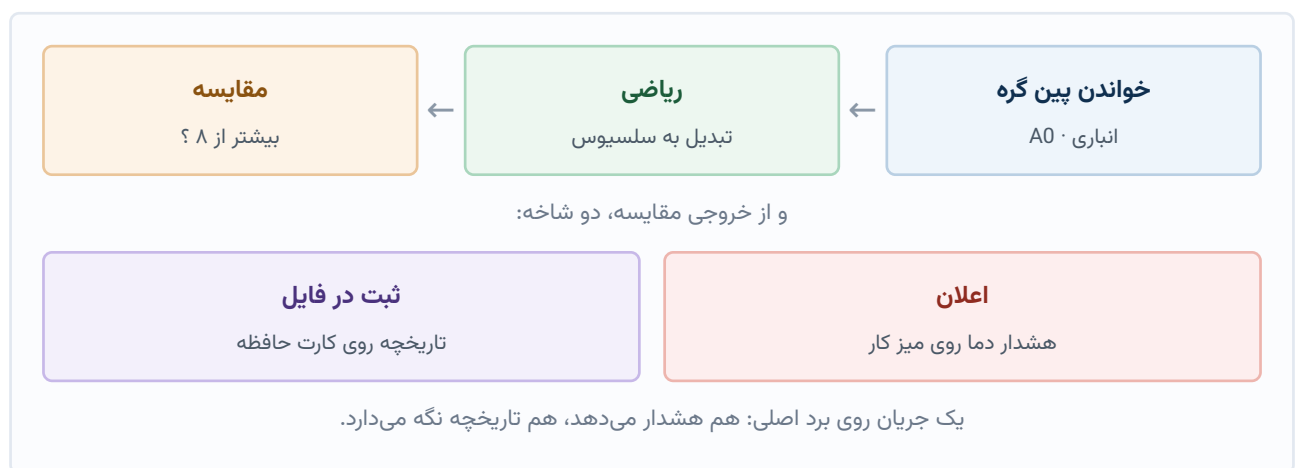
هر بار که حرکتی دیده شود، تایمر دو دقیقه‌ای از نو شروع می‌شود. اگر کسی روی پله ایستاده و حرف می‌زند، چراغ خاموش نمی‌شود. دو دقیقه بعد از آخرین حرکت، خودش خاموش می‌شود.

اگر می‌خواهید فقط شب‌ها کار کند، یک نود ساعت در دیبا منیجر اضافه کنید و با اگر/وگرنه ترکیبش کنید – این یکی روی برد اصلی اجرا می‌شود، چون گره ساعت ندارد.

سناریو ۴ – پایش دمای انباری یا سردخانه

مسئله: فریزر یا انباری که اگر دمایش بالا برود، محتویاتش از بین می‌رود.

سنسور دما را به A0 یک گره وصل کنید و یک قانون گزارش با حساسیت مناسب بگذارید. سپس در دیبا منیجر روی برد اصلی:



یک جریان روی برد اصلی: هم هشدار می‌دهد، هم تاریخچه نگه می‌دارد.

نود ثبت در فایل خط‌ها را دسته‌ای می‌نویسد، نه یکی یکی – چون هر نوشتن روی دستگاه یعنی خواندن، تغییر و نوشتن دوباره، و انجامش به ازای هر خط، فلش را اذیت می‌کند.

سناریو ۵ – دوربین در ورودی

یک ESP32-CAM کنار در بگذارید. با یک سنسور PIR روی یکی از پین‌هایش، هر بار که کسی نزدیک شد یک عکس روی کارت حافظه ذخیره کنید. نمای زنده هم از طریق برنامه‌ی دوربین در دسترس است.

محدودیت واقعی

یک ESP32-CAM دوربین در حرفه‌ای نیست. کیفیت تصویر متوسط است، در نور کم ضعیف است، و پخش زنده روی وای‌فای ضعیف قطع‌ووصل می‌شود. برای «ببینم کی زنگ زد» خوب است؛ برای امنیت جدی نه.

سناریو ۶ – کتابخانه‌ی رسانه‌ی خانگی

یک کارت حافظه‌ی ۶۴ گیگابایتی پر از موسیقی داخل برد بگذارید و **سرور رسانه** را روشن کنید. هر کسی روی شبکه‌ی خانه با یک مرورگر می‌تواند گوش بدهد – بدون نصب هیچ برنامه‌ای.

- پخش در همان صفحه، با امکان جلو و عقب بردن
- رمز اختیاری برای اینکه هر کسی به کتابخانه نرسد
- **سرویس است، نه صفحه** – برد را ری‌استارت کنید، دوباره خودش بالا می‌آید

جمع‌بندی: چه منطقی کجا اجرا شود

منطق	کجا	چرا
واکنش فوری به یک سنسور	روی گره	زمان واکنش، و کار کردن با خاموش بودن برد اصلی
ترکیب چند گره با هم	برد اصلی	گره فقط پین‌های خودش را می‌بیند
هر چیزی که به ساعت یا تاریخ نیاز دارد	برد اصلی	گره ساعت ندارد
هر چیزی که به اینترنت نیاز دارد	برد اصلی	گره به اینترنت نمی‌رسد
ثبت تاریخچه	برد اصلی	کارت حافظه آنجاست

۱۰ برق، باتری و مصرف

مصرف واقعی

حالت	ESP32	ESP8266
بیکار، وای‌فای روشن	حدود ۸۰ میلی‌آمپر	حدود ۷۰ میلی‌آمپر
ارسال فعال	تا ۲۴۰ میلی‌آمپر	تا ۲۰۰ میلی‌آمپر
خواب سبک	حدود ۱ میلی‌آمپر	حدود ۱ میلی‌آمپر
خواب عمیق	حدود ۱۰ میکروآمپر	حدود ۲۰ میکروآمپر

نتیجه‌ی عملی

برد اصلی را با آداپتور برق بگذارید – با وای‌فای همیشه روشن، باتری چند روز بیشتر دوام نمی‌آورد. گره‌ها هم در این نسخه بیدار می‌مانند تا ضربان بفرستند و فرمان بگیرند، پس آن‌ها هم بهتر است برق ثابت داشته باشند. باتری اینجا برای پشتیبان است، نه برای کار دائمی.

پایش باتری

اگر بردی را با باتری کار می‌اندازید، Dsp32 می‌تواند شارژ آن را نشان دهد. اما برد به‌خودی‌خود نمی‌داند باتری وصل است – باید سیم‌کشی را توصیف کنید.

مقسم ولتاژ

```

+--- R1 --- * --- R2 --- GND
|
+--- ADC پین
    
```

پین ADC حداکثر حدود ۳/۱ ولت را می‌خواند. یک باتری لیتیومی پر ۴/۲ ولت است، پس باید تقسیم شود. با ولتاژ نصف می‌شود: ۲/۱ ولت، که راحت داخل محدوده است و جریان مصرفی مقسم حدود ۲۱ میکروآمپر است.

$$R1 = R2 = 100k$$

محاسبه به صورت زنده نشان داده می شود

در

تنظیمات ← باتری

همین طور که مقاومت ها را تایپ می کنید، حساب می شود که یک پک پر چند ولت روی پین می گذارد. اگر سیم کشی ای بدهید که از سقف ADC رد شود، با همان اعداد رد می شود – چون در آن حالت گیج روی یک عدد ثابت به ظاهر معقول گیر می کند و اصلاً متوجه نمی شوید که خطاست.

تنظیمات

فیلد	توضیح
پین حسگر	فقط پین های ADC1 – ADC2 با رادیوی وای فای تداخل دارد
R2 و R1	مقدار واقعی مقاومت ها بر حسب اهم
سلول های سری	معمولاً ۱
شیمی باتری	لیتیوم یون، لیتیوم پلیمر، یا LiFePO4
خالی و پر	ولتاژ هر سلول در دو سر بازه
هشدار و بحرانی	درصدهایی که در آن ها اعلان یا اقدام انجام شود

درصد شارژ از **درون یابی منحنی دشارژ واقعی** هر شیمی به دست می آید، نه از یک خط صاف بین خالی و پر. منحنی در میانه ی بازه تقریباً صاف است – دقیقاً همان جایی که دقت می خواهید – و خط صاف آنجا خطای بزرگی می دهد.

پایش در تردی جداگانه با اولویت پایین اجرا می شود و هر پنج ثانیه یک بار میانه ی شانزده نمونه را می گیرد. میانه، نه میانگین: کنار یک رادیوی روشن، مبدل آنالوگ گاهی عدد پرت می دهد و میانگین آن را نگه می دارد در حالی که میانه دورش می اندازد.

در سطح بحرانی، برد می تواند فقط اعلان بدهد، به ۸۰ مگاهرتز با خواب سبک برود، یا برای محافظت از سلول ها به خواب عمیق برود.

۱۱ راه اندازی گام به گام

گام ۱ – فلش کردن برد اصلی

تصویر آماده را از مخزن انتشار بگیرید. برای هر برد یک فایل **merged** هست که با یک دستور روی آفست صفر می‌رود:

```
pip install esptool

esptool.py --chip auto -p /dev/ttyUSB0 -b 460800 write_flash 0x0 \
firmware/esp32s3/dsp32-esp32s3-merged.bin
```

یا از اسکریپت همراه استفاده کنید که پورت و تراشه را خودش پیدا می‌کند:

```
./flash.sh esp32s3
```

گام ۲ – اولین اتصال

1. برد را به برق بزنید و حدود ده ثانیه صبر کنید.
 2. با گوشی به شبکه‌ی وای‌فای **Dibachain** وصل شوید.
 3. مرورگر را باز کنید و به <http://10.3.2.1> بروید. معمولاً پورتال کپتیو خودش پیشنهاد می‌دهد.
- جادوگر راه اندازی در چهار گام شما را می‌برد: معرفی، اتصال به وای‌فای خانه (اختیاری)، انتخاب برنامه‌ها، و نصب.

اتصال به وای‌فای خانه اختیاری است

اگر برد را به شبکه‌ی خانه وصل کنید، می‌توانید بدون تعویض وای‌فای گوشی به آن برسید و برنامه‌ها هم قابل دانلود می‌شوند. اگر وصل نکنید، همه چیز باز هم کار می‌کند – فقط باید هر بار به شبکه‌ی خود برد وصل شوید و برنامه‌ها را دستی از کارت حافظه نصب کنید.

گام ۳ – امن کردن

1. **تنظیمات ← شبکه** – نام و رمز اکسس‌پوینت را عوض کنید. این مهم‌ترین کاری است که می‌کنید.
2. **تنظیمات ← حساب کاربری** – یک حساب بسازید و رمز بگذارید. رمز روی خود برد ذخیره می‌شود، پس از هر گوشی‌ای که وصل شوید همان حساب هست.
3. اگر کارت حافظه دارید، حساب و داده‌ها روی کارت می‌روند – که از فلش کردن دوباره‌ی برد جان سالم به در می‌برند.

گام ۴ – اولین گره

1. یک ESP8266 بگیرید و چهار سیم را طبق فصل ۷ وصل کنید.
2. در برنامه‌ی **Dmesh** دکمه‌ی **+** را بزنید، پین‌ها را انتخاب کنید و منتظر بمانید. حدود یک دقیقه طول می‌کشد.
3. گره خودش را اعلام می‌کند و در فهرست ظاهر می‌شود.
4. **Claim** بزنید تا کلید مخصوص خودش را بگیرد.
5. پین‌ها را تنظیم کنید و قوانین را بسازید.

بعد از این، دیگر کابل لازم نیست

هر به‌روزرسانی بعدی گره از راه دور انجام می‌شود. کابل USB دقیقاً یک بار برای هر برد لازم است.

گام ۵ – پشتیبان‌گیری

چیزهایی که ارزش نگه داشتن دارند:

- **جریان‌های دیبا منیجر** – در پوشه‌ی `/flash/Flows` یا روی کارت. با مدیر فایل داندلودشان کنید.
- **تنظیمات گره‌ها** – روی خود گره‌ها در EEPROM هستند، پس با فلش کردن دوباره‌ی برد اصلی از بین نمی‌روند. اما کلیدها روی برد اصلی‌اند: اگر برد اصلی را پاک کنید، باید گره‌ها را دوباره تصاحب کنید.
- **حساب کاربری** – فایل `Dsp32/account.json` روی کارت یا فلش.

محدودیت‌ها و آنچه نباید انتظار داشت ۱۲

هر سامانه‌ای مرز دارد. آنچه این فصل را ارزشمند می‌کند این است که مرزها را پیش از خرید و سیم‌کشی بدانید، نه بعد از آن.

مرزهای سخت‌افزاری

محدودیت	توضیح
حافظه‌ی حدود ۳۲۰ کیلوبایت	هیچ‌وقت نمی‌شود فایل بزرگی را کامل در حافظه باز کرد. همه چیز جریانی خوانده می‌شود.
ADC2 غیرقابل استفاده	با رادیوی وای‌فای تداخل دارد. فقط پین‌های ADC1 برای خواندن آنالوگ.
بدون شتاب‌دهنده‌ی گرافیکی	همه‌ی رابط کاربری در مرورگر شماست، پس این مسئله نیست – ولی برد خودش چیزی رسم نمی‌کند.
یک اکسس‌پوینت با ظرفیت محدود	حدود ۴ تا ۸ دستگاه هم‌زمان. این یک روتر نیست.

مرزهای شبکه‌ی گره‌ها

- **هشت قانون به ازای هر گره.** این ظرفیت EEPROM گره است.
- **مسیریابیِ مِش ندارد** با وجود نامش. هر گره مستقیم با برد اصلی حرف می‌زند؛ گره‌ها برای هم رله نمی‌کنند. گره‌ای که بیرون از برد رادیویی باشد، بیرون از دسترس است.
- **تصاحب، اعتماد در اولین برخورد است.** در دو دقیقه‌ی اول بعد از روشن شدن گره، و روی شبکه‌ای که در اختیار خودتان است.
- **رجیستری سقف ثابتی ندارد،** اما حافظه‌ی NVS برد حدود دویست گره را می‌کشد.

مرزهای نرم‌افزاری

- برنامه‌ها از هم جدا نیستند**

یک برنامه‌ی نصب‌شده جاوااسکریپت است که در همان صفحه‌ی میز کار اجرا می‌شود. یک برنامه‌ی بدخواه می‌تواند به هر چیزی که میز کار دسترسی دارد برسد. فقط بسته‌هایی را نصب کنید که به منبعشان اعتماد دارید.
- **مرورگر وب ندارد.** نمی‌شود با آن اینترنت گشت. دلیلش حافظه است: یک موتور رندر مدرن چند ده مگابایت می‌خواهد.

- **پخش ویدیوی سنگین نه.** سرور رسانه فایل را می‌فرستد و مرورگر شما پخشش می‌کند؛ برد چیزی را تبدیل نمی‌کند. یک MKV با کدک عجیب دانلود می‌شود ولی پخش نمی‌شود.
- **سرعت، سرعت کارت و رادیو است، نه برد** — چند مگابایت بر ثانیه.

آنچه هنوز نیست

قابلیت	وضعیت
HTTPS روی خود برد	نیست. یک نشست TLS حدود ۴۵ کیلوبایت از حافظه می‌خواهد که با بقیه‌ی کارها جمع نمی‌شود.
جداسازی واقعی برنامه‌ها	نیست. نیاز به اجرای هر برنامه در قاب مجزا دارد.
خواب عمیق گره‌ها با باتری	نیست. گره برای ضربان و فرمان‌گیری بیدار می‌ماند.
رله‌ی گره به گره	نیست. همه مستقیم با برد اصلی.
کنترل از بیرون خانه	نیست، و عمداً. چیزی به بیرون باز نمی‌شود.

مقایسه با گزینه‌های دیگر ۱۳

Dsp32 جایگزین همه چیز نیست. این جدول کمک می‌کند بدانید کی انتخاب درستی است و کی نیست.

گزینه	قوت	کی Dsp32 بهتر است
Home Assistant روی رزبری پای	پشتیبانی از هزاران دستگاه، جامعه‌ی بزرگ، رابط پخته	وقتی نمی‌خواهید یک کامپیوتر کامل با کارت SD ای که می‌سوزد و مصرف چند وات داشته باشید. Dsp32 حدود یک بیستم برق مصرف می‌کند و روی قطعه‌ای چند دلاری اجرا می‌شود.
دستگاه‌های هوشمند تجاری	از جعبه کار می‌کنند، اپ آماده دارند	وقتی نمی‌خواهید داده‌های خانه‌تان روی سرور شرکتی برود، یا وقتی نمی‌خواهید محصولی بخرید که با تعطیل شدن شرکت بی‌فایده شود.
ESPHome و Tasmota	پخته، پایدار، هزاران دستگاه	وقتی رابط کاربری روی خود دستگاه می‌خواهید بدون یک کنترل‌کننده‌ی مرکزی جدا. ESPHome به Home Assistant نیاز دارد؛ Dsp32 خودش هر دو است.
Node-RED	قدرت و انعطاف بسیار بیشتر در ساخت منطق	وقتی نمی‌خواهید یک سرور جدا برای اجرای منطق داشته باشید. دیبا منیجر خیلی ساده‌تر است ولی روی خود برد اجرا می‌شود.

جمله‌ای که این پروژه را توصیف می‌کند

Dsp32 برای کسی است که می‌خواهد خانه‌اش هوشمند باشد، مالکیت کامل روی آن داشته باشد، و برای این کار نه سرور بخرد، نه اشتراک بدهد، نه به اینترنت وابسته باشد. در ازای این، باید کمی سیم‌کشی کند و کمی یاد بگیرد.

پیوست – مرجع فنی

واسط REST – مسیرهای اصلی

مسیر	کار
GET /api/system	تراشه، حافظه، دما، مدت روشن بودن
GET /api/fs/list?path=	فهرست یک پوشه
GET /api/fs/read?path=	خواندن یک فایل
POST /api/fs/write?path=	نوشتن یک فایل
GET /api/gpio/pins	نقشه‌ی توانایی همه‌ی پین‌ها
POST /api/gpio/config?pin=&mode=	تعیین حالت یک پین
POST /api/gpio/write?pin=&value=	راندن یک خروجی
GET /api/gpio/read?pin=	خواندن یک ورودی
GET /api/wifi/status	وضعیت اکسس‌پوینت و اتصال بالادست
GET /api/mesh/list	همه‌ی گره‌های ثبت‌شده
POST /api/mesh/claim?id=	تصاحب یک گره
POST /api/mesh/desired?id=	تعیین وضعیت مطلوب یک گره
POST /api/mesh/write?id=&pin=&value=	راندن یک پین روی گره
GET /api/battery/status	وضعیت باتری
GET /api/media/status	وضعیت سرور رسانه

حالت‌های پین

حالت	کاربرد
unused	رها - حالت امن
in	ورودی دیجیتال
in_pullup	ورودی با مقاومت بالاکش داخلی - برای کلید به زمین
out	خروجی دیجیتال - رله، LED
pwm	خروجی با عرض پالس متغیر - دیمر، سرعت موتور
adc	ورودی آنالوگ - فقط پین‌های ADC1

پورت‌ها

پورت	کار
80	میز کار و واسط REST
81	جریان زنده‌ی دوربین
53	پورتال کپیو (DNS)
8080	سرور رسانه (قابل تغییر)
32700	پروتکل Dmesh روی UDP

منابع

مورد	نشانی
مخزن انتشار	github.com/AliAkrami1375/Dsp32-firmware
شبیه‌ساز مرورگری	aliakrami1375.github.io/Dsp32-firmware/demo/
راهنمای فلش کردن	docs/FLASHING.fa.md
راهنمای Dmesh	docs/DMESH.fa.md
راهنمای سرور رسانه	docs/MEDIA_SERVER.fa.md
راهنمای ساخت برنامه	docs/APP_DEVELOPMENT.fa.md

این دفترچه بخشی از پروژه‌ی Dsp32 است و همراه نسخه‌ی ۱/۱/۲ منتشر شده.
هر جا که چیزی نادرست یا مبهم بود، در مخزن گزارش کنید.